Lany : décisions d'architecture

Synthèse des décisions validées aux étapes 1 à 3. À exporter en Markdown dans `docs/` du monorepo.

1. Vue d'ensemble

```
Navigateur → Next.js (BFF) → Spring Boot (API REST) → PostgreSQL
```

Application	Rôle
Spring Boot	Logique métier, calculs, accès base, autorisation par foyer. Source de vérité.
Next.js	Affichage, navigation, formulaires. Aucune règle métier.

Règle d'or : une règle métier change → on ne modifie que Spring.

2. Règles métier

Sujet	Décision
Reste principal	**Réel** : départ + report + entrées − dépenses (transactions réelles uniquement).
Simulation	Reste réel − fixes projetées non réalisées, d'aujourd'hui à la fin du mois.
Dépenses fixes	Règle modifiable, projetée par défaut ; « réalisée » manuellement = création d'une transaction (une par période).
Équivalence budgétaire	5 jours × 4 semaines, configurables par foyer. Page Dépenses fixes uniquement.
Projection	Calendrier réel, sans plafond. Journalière : jours budgétaires (lundi-vendredi par défaut).

Sujet	Décision
Semaine	ISO lundi-dimanche, rattachée au mois de son lundi.
Report de mois	Proposé, confirmé, puis figé. Déficit reporté tel quel.
Clôture de mois	Hors MVP, prévue dans la structure (`closed_at`).
Rôles	`owner` (créateur du foyer), `write`, `read`, par foyer.
Multi-foyers	Prévu dès la conception (`household_id` partout).

3. Base de données

Sujet	Décision
SGBD	PostgreSQL 15+ (15.13 en local)
Migrations	Flyway : `V1__init.sql` (schéma), `V2__add_version_columns.sql` (verrouillage optimiste)
Clés primaires	`BIGINT` identity
Montants	`NUMERIC(15,2)`, toujours positifs, `type` donne le sens
Instants	`TIMESTAMPTZ` (UTC), jour métier dérivé en `Indian/Antananarivo`
Énumérations	`VARCHAR` + `CHECK`
Intégrité entre foyers	Clés étrangères composites `(id, household_id)`
Anti-double-comptage	`EXCLUDE` (`btree_gist`) sur la période couverte
Suppression	Archivage (`archived_at`) plutôt que suppression ; membres jamais supprimés
Arrondi	`BigDecimal` pleine précision, `HALF_UP` à 2 décimales à l'enregistrement ou l'affichage uniquement

Sujet	Décision
Statistiques	Vue SQL pour les faits agrégés, Java pour les règles

4. Backend (Spring Boot, Maven)

Sujet	Décision
Organisation	Packages par fonctionnalité : `auth`, `household`, `member`, `category`, `budget`, `fixedexpense`, `transaction`, `dashboard`, `shared`
Dépendances	`Controller → Service → Repository`, jamais dans l'autre sens
Injection	Par constructeur
Calculs	Domaine pur sans Spring : `Money`, `Frequency`, `BudgetCalendar`, calculateurs. `Clock` et `ZoneId` injectés
Entités	Riches (comportement, peu de setters), jamais renvoyées au client
DTO	`record` Java, mapping manuel au début (MapStruct plus tard si utile)
Schéma	Flyway propriétaire, `ddl-auto=validate`, `open-in-view=false`, enums en `STRING`, `LAZY`
Tests	JUnit sur le domaine, Testcontainers PostgreSQL pour les Repository (pas de H2)
Accès foyer	Composant `HouseholdAccess` appelé explicitement ; 404 pour un non-membre
API	Préfixe `/api/v1`, foyer dans l'URL
Concurrence	Verrouillage optimiste (`@Version`) → 409

5. Frontend (Next.js, npm)

Sujet	Décision
Routing	`/h/[householdId]/...`, jour des Flux en paramètre d'URL, URL en anglais, interface en français
Organisation	`features/` par fonctionnalité, pages minces dans `app/`
Composants	Serveur par défaut, `"use client"` pour l'interactivité seulement, au plus bas de l'arbre
Accès API	Module serveur unique (`server-only`), Server Actions pour les écritures, Route Handlers si un besoin apparaît
Types	Générés depuis OpenAPI, après la première fonctionnalité
Formulaires	`useActionState` d'abord, puis React Hook Form + Zod
Montants	Chaînes JSON, aucun calcul côté frontend, formatage par `Intl.NumberFormat`
UI	Tailwind, shadcn/ui, Radix ; sobre, sans gradients systématiques

6. Authentification et sécurité

Sujet	Décision
Mécanisme	Spring Security + JWT court
Stockage	Cookie `httpOnly` , `Secure` , `SameSite=Lax` , posé par Next.js
Vérification	Middleware optimiste (cookie présent) + vérification réelle par Spring à chaque appel
Refresh token	Reporté jusqu'à besoin avéré

Sujet	Décision
Mots de passe	Hachage BCrypt ou Argon2

7. Gestion des erreurs et validation

Sujet	Décision
Format	Problem Details (RFC 9457) avec `code`, `errors` par champ, `traceId`
Messages	En français dans le frontend, associés au `code`
Codes HTTP	400, 401, 403 (`read` qui écrit), 404 (ressource ou non-membre), 409 (conflit métier), 422 (validation), 500 (message générique)
Spring	Exceptions métier + un seul `@RestControllerAdvice` ; contraintes nommées de la base traduites en 409
Validation	4 niveaux : Zod, Bean Validation, Service, base. Aucun niveau n'est retiré
Next.js	Erreurs attendues retournées via `useActionState`, inattendues vers `error.tsx`
Double soumission	Bouton désactivé pendant l'envoi ; idempotence reportée

8. Points à vérifier au moment de la création des projets

- Versions actuelles de Java LTS, Spring Boot, Next.js, React, Tailwind, shadcn/ui.
- Règles de cache de la version de Next.js retenue.
- Nom exact du fichier de middleware dans cette version de Next.js.
- Support de `ProblemDetail` dans la version de Spring retenue.

9. Pièges à ne pas oublier

- `redirect()` et `notFound()` lèvent une exception interne : ne pas les avaler dans un `try/catch`.
- Ne jamais compter uniquement sur le middleware pour protéger des données.
- Ne jamais renvoyer une Entity JPA au client.
- Ne jamais faire de calcul métier côté Next.js.
- `SET NULL (colonne)` est obligatoire sur les clés composites, sinon `household_id` serait aussi annulé.